

QCon
全球软件开发大会

尽在上下文： JoyCode 的企业级 AI Coding 实践

徐翔

京东科技 AI 架构师

负责 JoyCode AI Coding 架构设计、Harness 工程建设；过往经历：先后就职于快手、蚂蚁，专注 AI Coding 领域、图计算领域、图可视化领域、中台能力建设。

极客邦科技 2026 年会议规划

促进软件开发及相关领域知识与创新的传播



参会咨询

🕒 6月 📍 上海

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：6月26-27日

- AI Infra 系统工程
- 多 Agent 协作与实践
- 多模态融合
- 模型训练与推理创新
- 数据平台与特征服务

🕒 8月 📍 深圳

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：8月21-22日

- Agentic AI
- 轻量化与高效推理
- 多模态应用
- AI + IoT 场景实践
- AI 工业化落地

🕒 10月 📍 上海

👥 1200人

QCon

全球软件开发大会

会议时间：10月22-24日

- AI Agent
- Vibe Coding
- 智能可观测
- 推理基建
- 模型攻防
- AI x 创造力

🕒 12月 📍 北京

👥 1000人

AiCon

全球人工智能开发与应用大会

会议时间：12月18-19日

- 大模型架构创新
- 多模态 AI 产业融合
- 具身智能
- AI for Science
- 大模型安全

目录

01 企业在编码场景的核心挑战

02 JoyCode 的创新解决方案

03 未来趋势：尽在上下文

01

企业在编码场景的核心挑战

企业在编码场景面临的核心挑战

京东作为电商巨头，业务场景极其复杂，涉及零售、物流、金融、健康等多个专业领域。大量自研框架、中间件和业务组件构成了独特的技术生态，这对AI编码工具提出了极高的专业领域知识要求。

业务场景复杂

从初级开发工程师到资深架构师，从业务开发到基础架构，从敏捷团队到传统瀑布模型，京东内部用户群体呈现出高度多元化的特征。不同角色对AI编码工具的期望和使用方式存在显著差异，如何满足多样化的需求成为一大挑战。

用户画像丰富

企业级开发任务往往涉及跨模块、跨系统的复杂变更，单次任务可能需要理解数千个文件、数十个服务。这种复杂性导致上下文迅速膨胀，远超模型的处理能力，形成“上下文爆炸”的困境。

复杂长任务

如庞大的代码海洋中，如何快速定位相关代码、理解业务逻辑、避免重复造轮子，成为AI编码工具必须解决的核心问题。

超大代码仓

02

JoyCode 的解决方案

企业知识增强

JoySpace

在线文档、企业知识增强

RepoWiki

“代码在哪里，知识就在哪里”



企业知识

JoyAgent & JoyContext

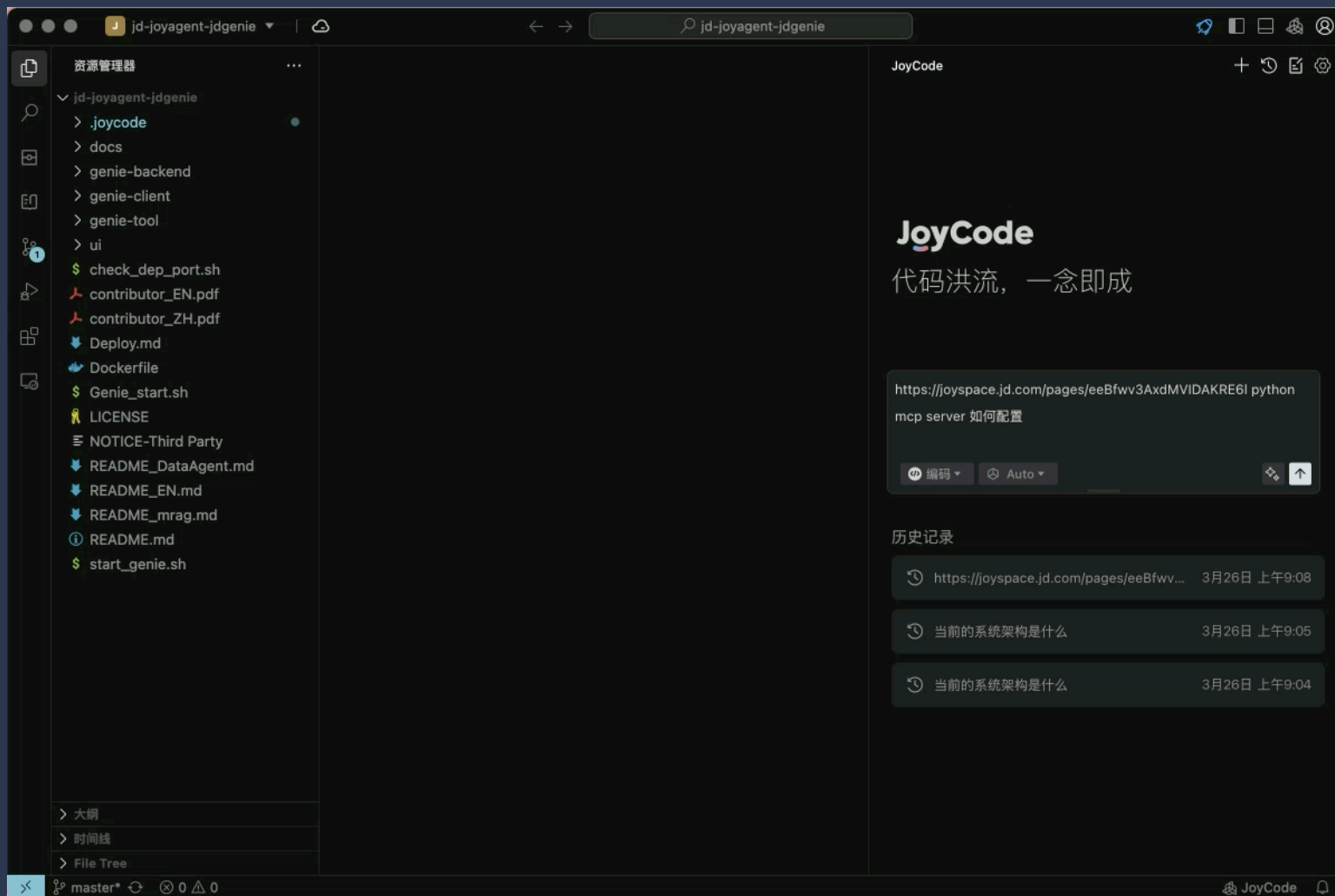
多模态非结构化知识增强

DongDesign & Relay

设计稿到企业 UI 组件通路

在线文档知识增强

企业在线文档内容获取。



多模态知识增强

JoyContext 为 JoyCode 提供多模态知识检索服务，通过内置的知识库查询工具实现上下文增强。

JoyAgent 则在多模态知识之上，提供了更加丰富的能力，用户可以自定义智能体，配置个性化的 MCP Skills、工作流等。



JoyAgent workflow编排

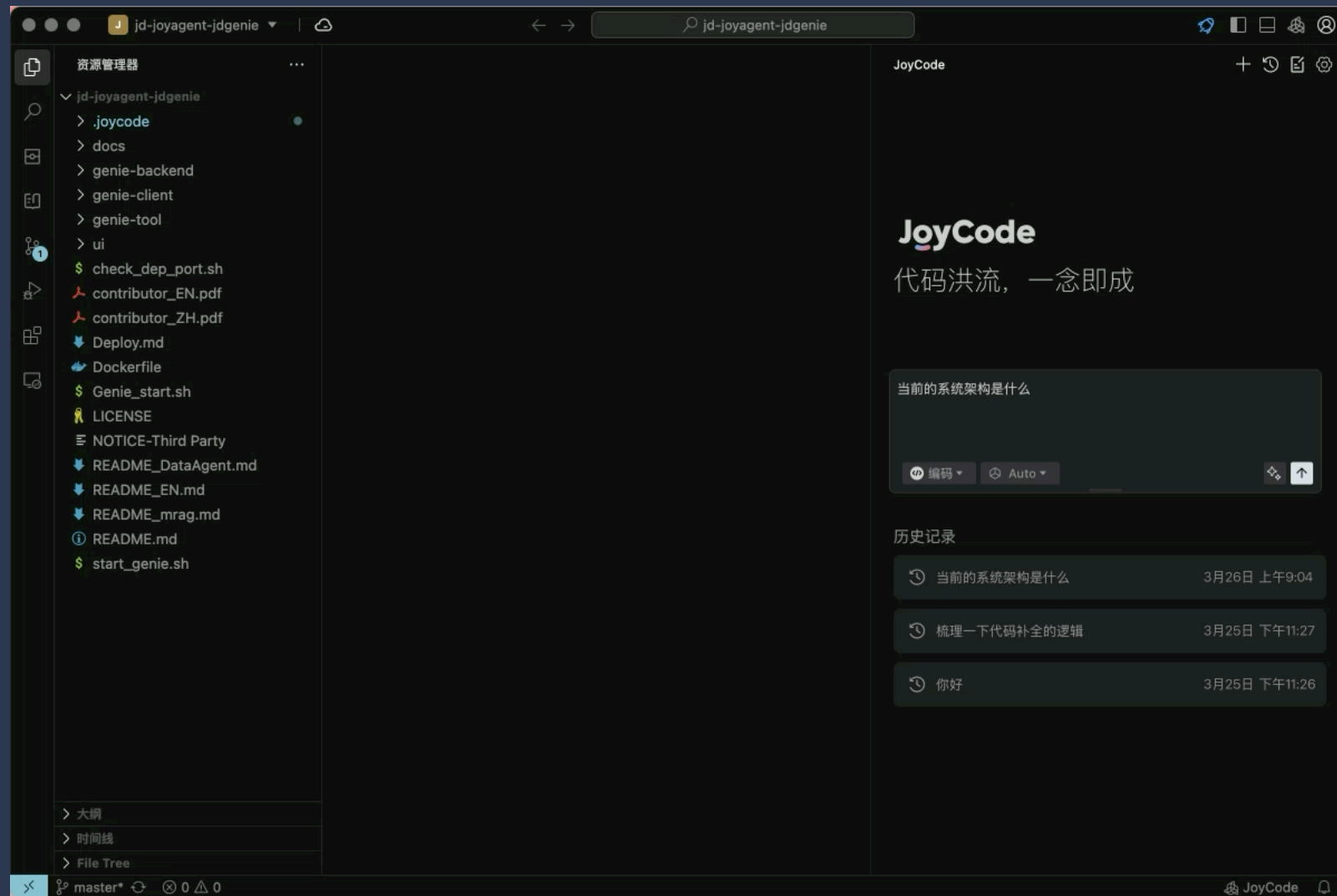


JoyContext 多模态检索



仓库代码知识增强

CodeWiki 通过AI能力生成仓库 Wiki，形成清晰、结构化、「文字+图片」形式的知识体系，无需人工逐行梳理，让文档内容、可视化架构、源码交互一目了然，具备发生变化即时更新的能力。



仓库代码知识增强

REPO WIKI

▼ 生成状态

✅ 已完成
已完成: 20/20, 失败: 0

▼ 目录

项目概述

▼ 核心业务模块

多智能体系统

智能问答系统

工具集成系统

对话交互系统

文件管理系统

快速上手指南

▼ 系统架构设计

多智能体框架

数据处理架构

工具生态系统

通信协议设计

▼ 技术实现细节

大语言模型集成

向量搜索与 RAG

数据库抽象层

前端架构实现

▼ 集成模式与扩展

API设计与使用

自定义工具开发

部署与运维

监控与日志

您觉得 Repo Wiki 是否有帮助?

← → jd-joyagent-jdgenie 更新

多智能体系统 快速上手指南 大语言模型集成 API设计与使用 多智能体框架 ×

多智能体框架

架构概述

JoyAgent多智能体框架是一个基于Java的企业级智能代理系统, 采用分层架构设计, 支持多种智能体类型的协同工作。该框架通过抽象化的智能体基类和灵活的工具调用机制, 实现了可扩展的多智能体协作模式。

核心设计理念

框架遵循“分层抽象、职责分离、工具驱动”的设计原则, 通过BaseAgent抽象基类定义智能体的核心行为模式, 不同类型的智能体继承并实现特定的业务逻辑。系统采用ReAct (Reasoning and Acting) 模式作为智能体的基础执行范式, 确保智能体具备推理和行动的双重能力¹。

系统架构

```
graph TD; Client[客户端请求] --> Genie[GenieController]; Genie --> MultiAgent[MultiAgentService]; MultiAgent --> AgentFactory[AgentHandlerFactory]; AgentFactory --> PlanningAgent[PlanningAgent]; AgentFactory --> ExecutorAgent[ExecutorAgent]; AgentFactory --> ReActAgent[ReActAgent]; PlanningAgent --> Tools[工具集合]; ExecutorAgent --> Tools; ReActAgent --> Tools; ReActAgent --> Memory[记忆管理]; Tools --> PlanningTool[PlanningTool]; Tools --> CodeInterpreter[CodeInterpreterTool]; Tools --> FileTool[FileTool]; Tools --> DeepSearch[DeepSearchTool]; Tools --> LLM[大语言模型]; Memory --> LLM;
```

仓库代码知识增强

REPO WIKI

生成状态

已完成
已完成: 20/20, 失败: 0

目录

项目概述

核心业务模块

多智能体系统

智能问答系统

工具集成系统

数据分析系统

对话交互系统

文件管理系统

快速上手指南

系统架构设计

多智能体框架

数据处理架构

工具生态系统

通信协议设计

技术实现细节

大语言模型集成

向量搜索与RAG

数据库抽象层

前端架构实现

集成模式与扩展

API设计与使用

自定义工具开发

部署与运维

监控与日志

您觉得 Repo Wiki 是否有帮助?

jd-joyagent-jdgenie

更新

多智能体系统

快速上手指南

快速上手指南

系统概述

Genie AI智能体系统是一个基于多模块架构的智能对话平台,集成了大语言模型 (LLM)、数据分析、代码解释器和多种工具调用能力。系统采用前后端分离架构,支持多智能体协作,能够处理复杂的任务规划和执行。

核心架构

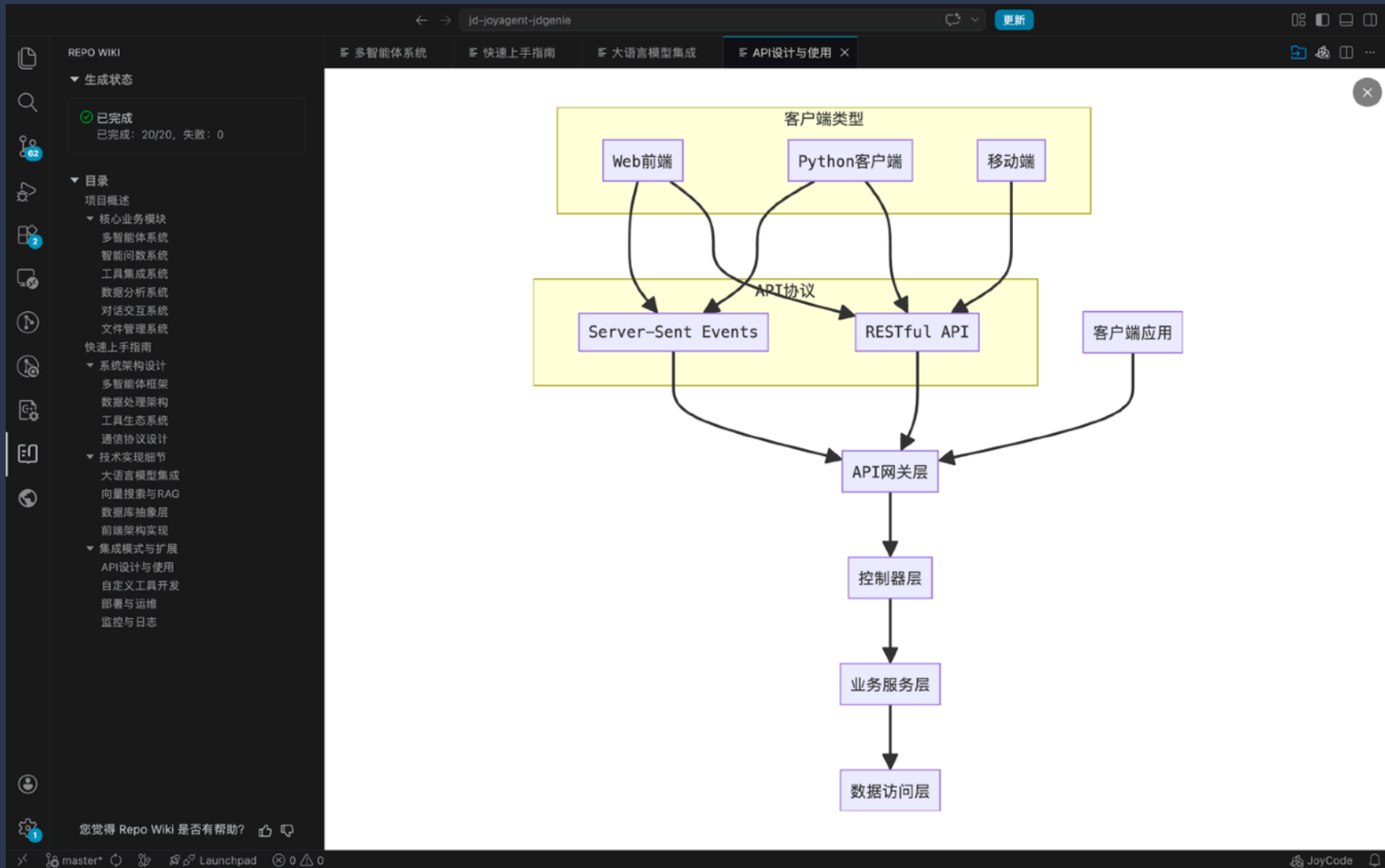
系统由四个主要模块组成:

1. **genie-backend** - Spring Boot后端服务,提供核心API和智能体协调
2. **genie-client** - Python客户端,实现MCP (Model Context Protocol) 服务
3. **genie-tool** - 工具服务模块,提供数据分析、搜索、代码执行等功能
4. **ui** - React前端界面,提供用户交互界面

```
graph TD; UI[前端界面 UI] --> Backend[后端服务 genie-backend]; Backend --> MCP[MCP客户端 genie-client]; Backend --> Tool[工具服务 genie-tool]; Backend --> H2[H2数据库]; MCP --> LLM[大语言模型]; Tool --> Vector[向量数据库]; Tool --> Search[搜索引擎];
```

图1: Genie系统架构图 (类型:系统架构图)

仓库代码知识增强



CodeWiki 的核心价值

痛点

① **知识传递**

 知识传递不畅，新成员上手难

 培训成本高，适应周期长



解决方案

知识传递

 基于代码整合项目背景、技术架构

 缩短适应周期，降低培训成本

② **技术债治理**

 历史技术债积累，代码结构复杂

 依赖关系混乱，维护效率低



技术债治理

 精准识别核心代码链路

 可视化呈现架构与模块调用关系

③ **AI代码生成**

 AI生成代码难符项目规范

 需要人工反复调优，错误率高



AI代码生成

 基于结构化知识赋能AI

 自动生成代码贴合项目规范

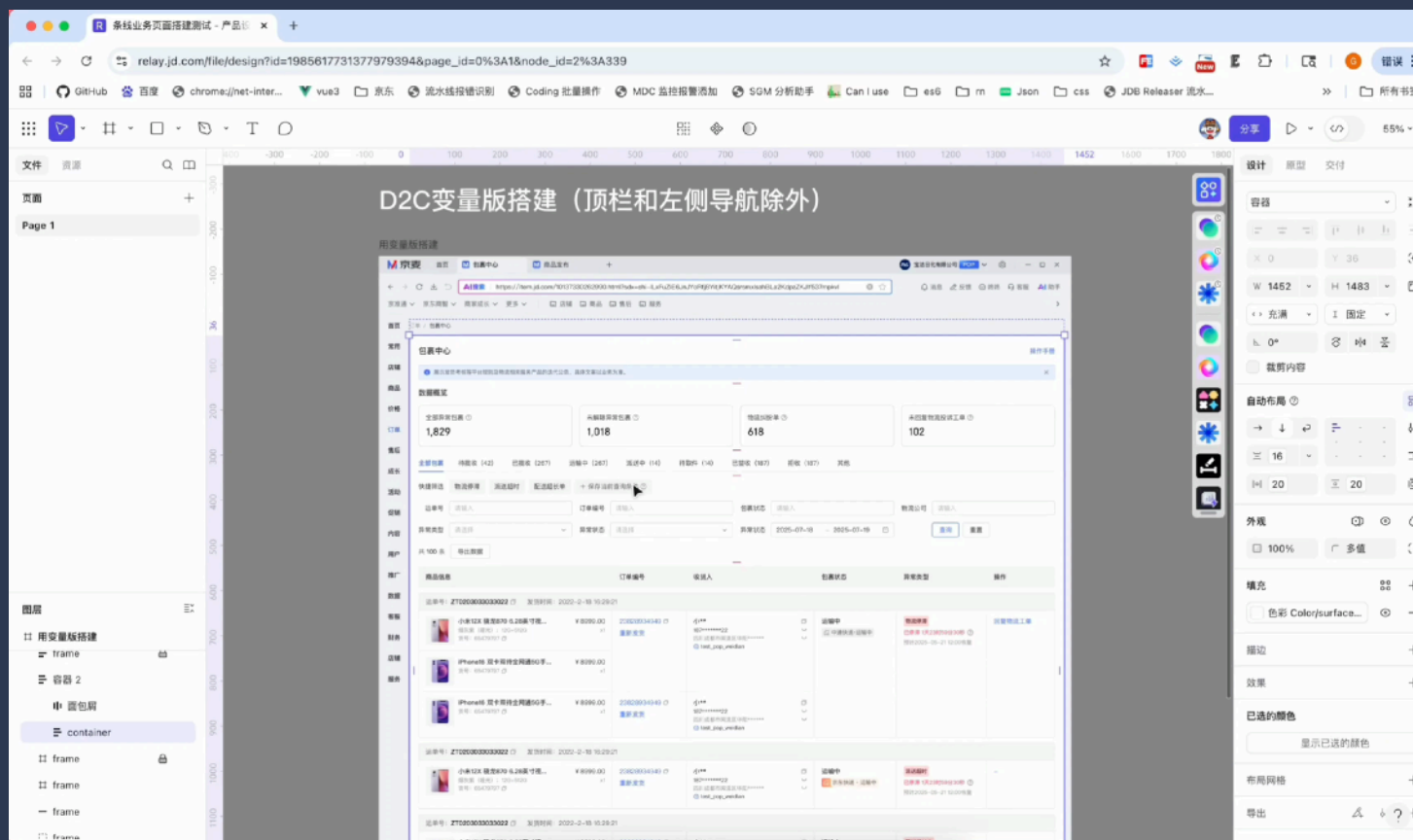
D2C-设计&企业组件知识增强

Design to Code 即设计稿转代码

DongDesign 是京东内部的组件生态平台。

Relay 是京东内部的设计生态平台。

JoyCode 通过 D2C MCP Server 实现企业级前端代码生成。



以检索为核心



1

超大代码仓，模型如何理解

提供丰富地检索工具，把选择权交给模型



2

先定位，再读取

提供代码行数、片段，减少上下文注入

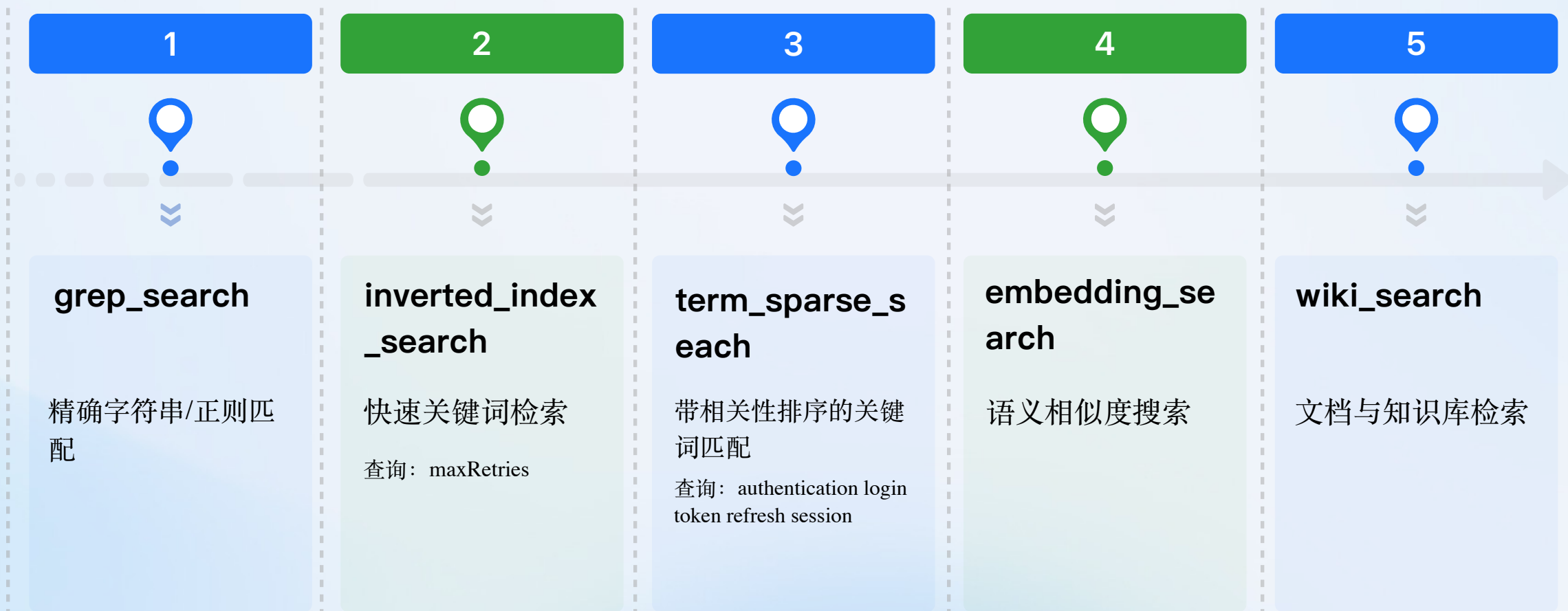


3

提供预处理仓库级知识

Wiki、代码图索引提前构建

多路检索



grep_search

优势:

- ✓ 精确性高: 正则表达式精确匹配, 结果无歧义
- ✓ 性能优秀: ripgrep优化, 响应速度较快
- ✓ 灵活性强: 支持复杂正则表达式模式
- ✓ 无索引依赖: 无需预先构建索引, 即搜即用
- ✓ 上下文丰富: 提供匹配行前后上下文

劣势:

- ✗ 无语义理解: 无法理解查询的语义含义
- ✗ 模式依赖: 需要精确的模式或关键词
- ✗ 模糊查询弱: 对模糊描述无能为力
- ✗ 大型仓库慢: 在超大代码库中可能较慢

查询示例: 搜索 `class\s+\w+`、查找 `API_KEY` 配置项

embedding_search (语义检索)

优势:

- ✓ 智能理解: 理解自然语言描述, 不依赖精确关键词
- ✓ 相似度匹配: 找到概念相似的代码, 即使关键词不同
- ✓ 跨语言支持: 不依赖特定编程语言语法
- ✓ 模糊查询强: 适合不确定具体实现名称的场景
- ✓ 概念性强: 适合高层次的概念性搜索

劣势:

- ✗ 索引耗时: 需要预先建立索引, 首次使用较慢
- ✗ 外部依赖: 依赖远程服务, 需要网络连接
- ✗ 精度损失: 可能有轻微的精度损失
- ✗ 存储开销: 索引占用存储空间
- ✗ 不适合精确查找: 精确查找不如grep

查询示例: "用户认证流程" "错误处理机制" "排序算法实现", 自然语言查询、模糊搜索

优势:

- ✓ 专注文档: 专注于项目Wiki文档内容
- ✓ 结构化: 提供清晰的文档结构
- ✓ 高层次: 适合理解高层次概念和设计

劣势:

- ✗ 范围有限: 仅限于Wiki内容, 不搜索代码
- ✗ 不搜索实现: 不包含代码实现细节
- ✗ 依赖文档质量: 结果质量依赖Wiki维护

查询示例: "用户认证架构设计" "性能优化最佳实践" "API 调用指南" "项目规范"

inverted_index_search (倒排索引搜索)

优劣势：

- ✓ 灵活匹配：只要匹配任意一个关键词就能找到结果
- ✓ 响应速度快：基于预先建立的索引，查询迅速
- ✓ 适合专业术语：对确定的技术术语效果最好
- ...
- ✗ 需要索引：依赖预计算的倒排索引
- ✗ 无语义理解：纯粹的关键词匹配
- ...

核心原理：

预先建立关键词到代码块的映射表，就像书籍的目录索引一样。

关键词来源：代码中的函数名、类名、变量名等

查询时：快速查找包含关键词的所有代码块

排序规则：匹配的关键词越多，排名越靠前

查询示例：“auth token login” “UserService UserDao” 任一关键词匹配

term_parse_search (稀疏检索)

优势:

- ✅ 结果更准确: 让最相关的结果排在前面
- ✅ 多词同时出现: 优先返回同时包含多个查询词的代码块
- ✅ 精确匹配: 适合查找包含特定词汇组合的代码
- ...
- ❌ 依赖关键词提取: 关键词提取的质量直接影响搜索效果
- ❌ 不理解语义: 无法理解代码的实际含义, 只做字面匹配

核心原理:

关键词来源: 代码中的函数名、类名、变量名等

计算文档相关性分数, 综合考虑以下因素:

- 词的重要性 (常见词权重低, 稀有词权重高)
- 词的出现频率 (出现越多越相关, 但有上限)
- 多个查询词是否同时出现

与倒排索引的区别

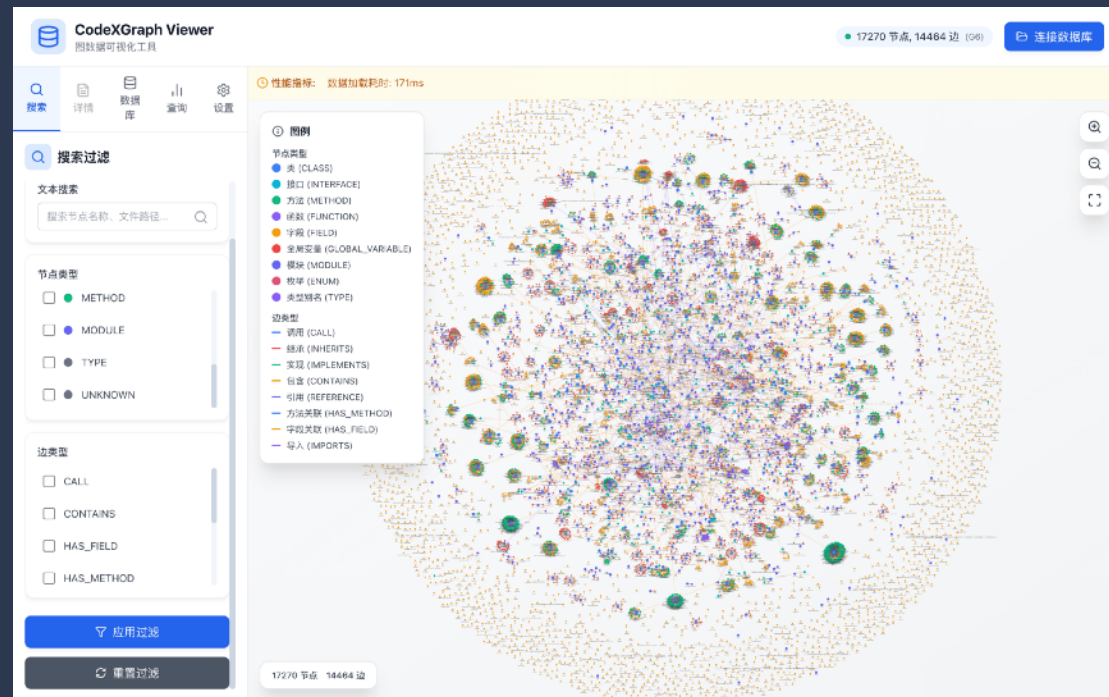
InvertedIndexSearch 适合快速定位的场景, 能够快速返回包含目标关键词的文档;

TermSparseSearch 更适合需要相关性排序的场景, 能够按相关度高低呈现更精准的结果。

查询示例: “user token validate” “fetch request response” 词项共现关系

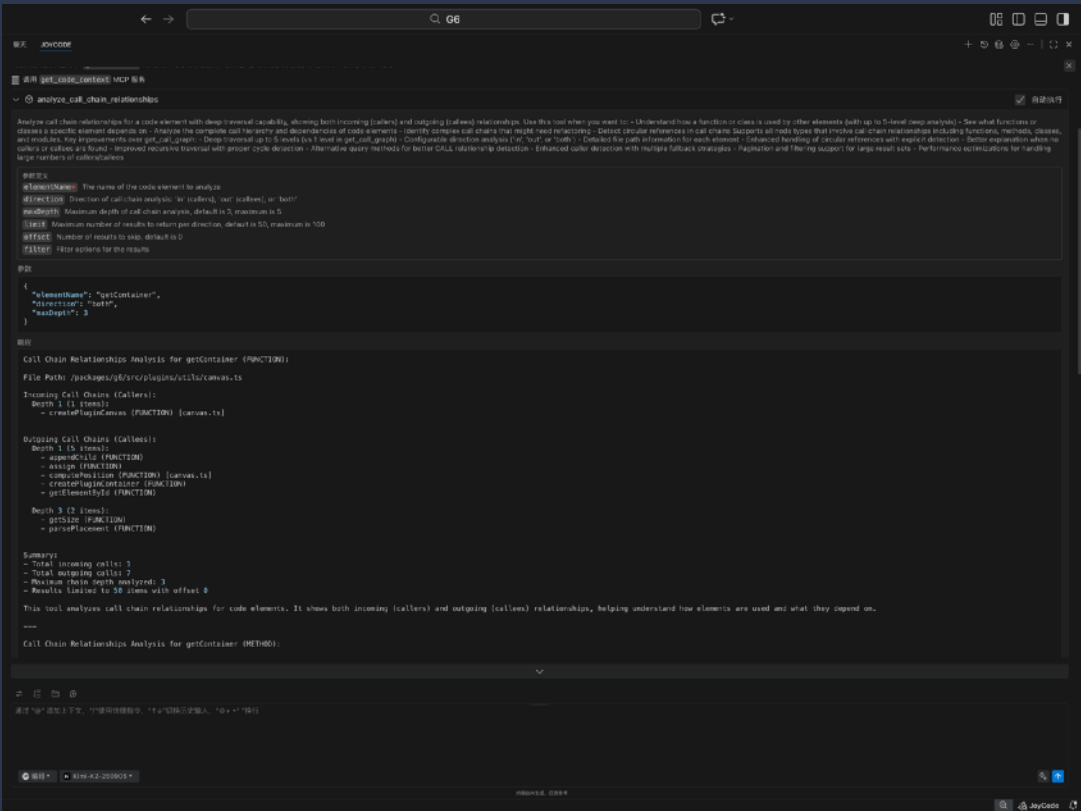
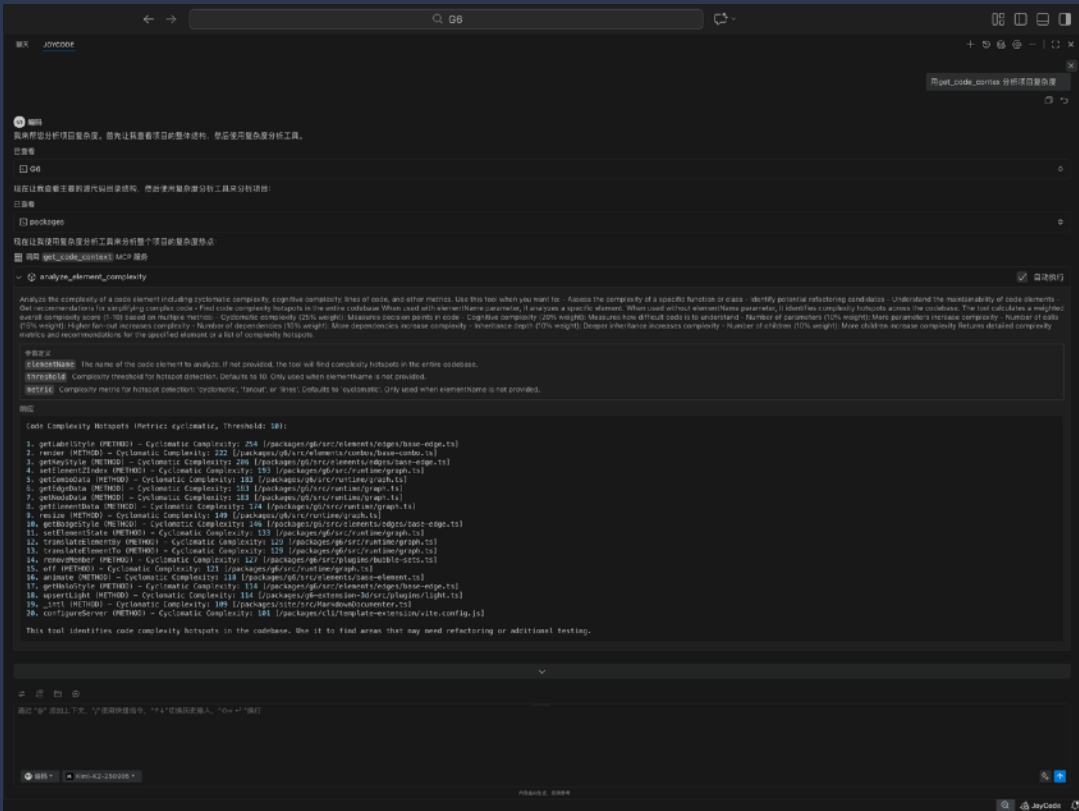
大型代码仓上下文理解

通过代码图索引实现对大型代码仓，针对仓库级理解，除了 CodeWiki 还为模型提供 Code Graph Search



大型代码仓上下文理解

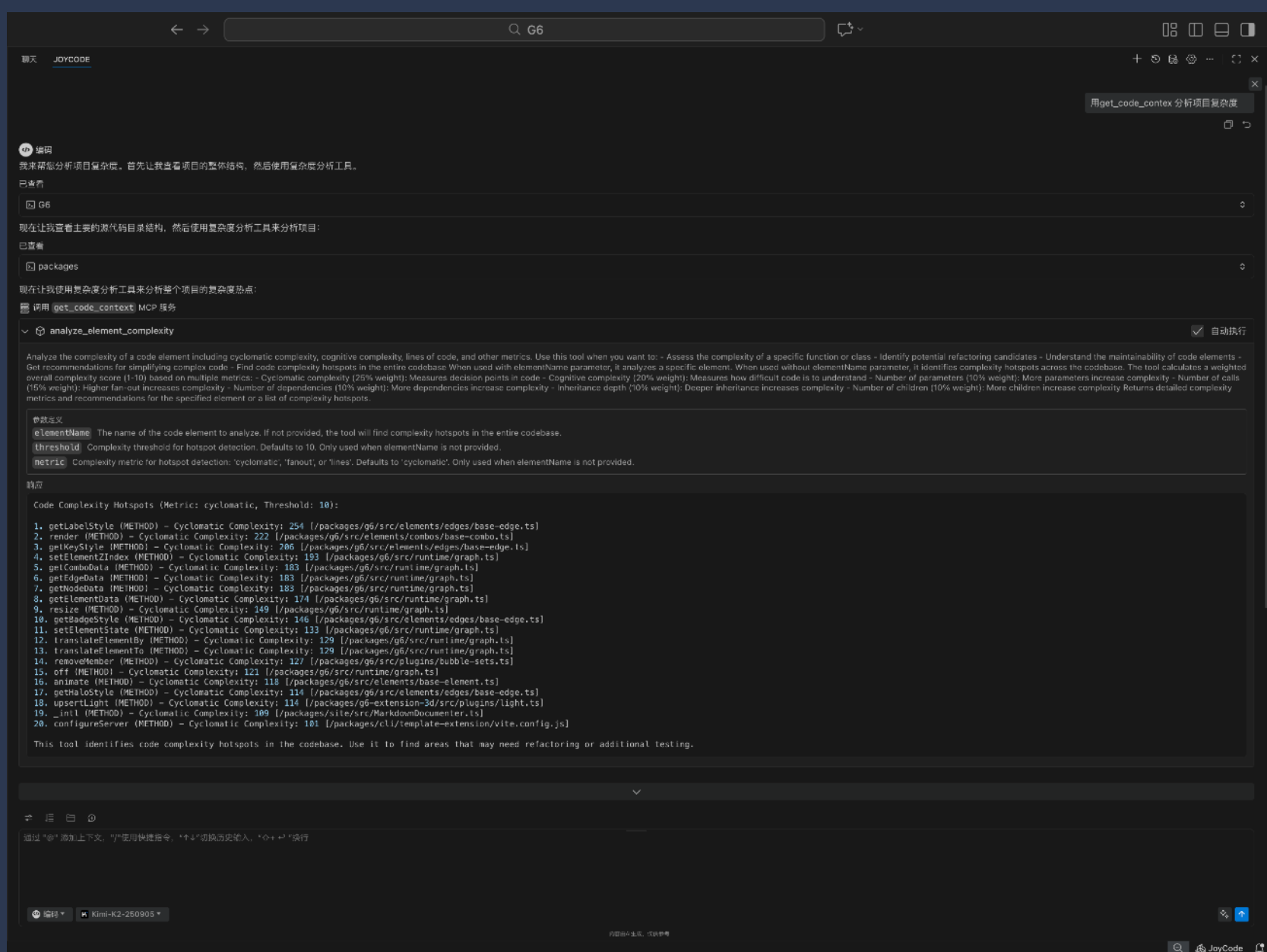
通过代码图索引实现对大型代码仓，仓库级理解：为模型提供全局理解的工具，如项目复杂度、调用链等



项目复杂度分析

通过圈复杂度、参数数量、调用数量、依赖数量、继承深度等多维度加权评分

可用于重构、项目优化等场景



函数调用链路分析

查看代码变更传播影响范围、循环引用等

The screenshot displays the G6 MCP service interface, which provides a deep traversal capability for analyzing call chain relationships. The interface includes a search bar at the top with the text "G6". Below the search bar, there is a section titled "analyze_call_chain_relationships" with a checkbox for "自动执行" (Auto Execute). The main content area shows the analysis results for the "getContainer" function, including incoming and outgoing call chains, a summary, and a detailed description of the tool's capabilities.

参数定义

- elementName**: The name of the code element to analyze
- direction**: Direction of call chain analysis: 'in' (callers), 'out' (callees), or 'both'
- maxDepth**: Maximum depth of call chain analysis, default is 3, maximum is 5
- limit**: Maximum number of results to return per direction, default is 50, maximum is 100
- offset**: Number of results to skip, default is 0
- filter**: Filter options for the results

参数

```
{  "elementName": "getContainer",  "direction": "both",  "maxDepth": 3}
```

响应

Call Chain Relationships Analysis for getContainer (FUNCTION):

File Path: /packages/g6/src/plugins/Utils/canvas.ts

Incoming Call Chains (Callers):

Depth 1 (1 items):

- createPluginCanvas (FUNCTION) [canvas.ts]

Outgoing Call Chains (Callees):

Depth 1 (5 items):

- appendChild (FUNCTION)
- assign (FUNCTION)
- computePosition (FUNCTION) [canvas.ts]
- createPluginContainer (FUNCTION)
- getElementById (FUNCTION)

Depth 3 (2 items):

- getSize (FUNCTION)
- parsePlacement (FUNCTION)

Summary:

- Total incoming calls: 1
- Total outgoing calls: 7
- Maximum chain depth analyzed: 3
- Results limited to 50 items with offset 0

This tool analyzes call chain relationships for code elements. It shows both incoming (callers) and outgoing (callees) relationships, helping understand how elements are used and what they depend on.

Call Chain Relationships Analysis for getContainer (METHOD):



模型检索策略

概念探索策略

场景：不确定具体实现

EmbeddingSearchTool



GrepSearchTool

说明：自然语言定位 → 精确定位代码

关键词定位策略

场景：有多个关键词，需快速筛选

InvertedIndexSearchTool



TermSparseSearch

说明：快速筛选 → BM25排序

全面理解策略

场景：需全面理解功能模块

WikiSearchTool



GraphSearchTool



EmbeddingSearchTool

说明：设计文档 → 结构关系 → 实现代码

精确定位策略

场景：已知精确信息

GrepSearchTool



GraphSearchTool

说明：精确定位 → 结构分析

案例分析

项目

<https://github.com/continuedev/continue>

文件数: 2600+

代码行数: 64w+

模型: GLM5(thinking)



竞品对比

查找"自动补全功能的触发逻辑" 完成任务的时候列出各个工具调用的次数, 以及每次调用的原因

Searched for 5 patterns, read 6 files (ctrl+o to expand)

根据已读取的文件, 我来总结自动补全功能的触发逻辑:

自动补全触发逻辑概述

1. 触发入口 (`completionProvider.ts`)

VS Code 通过 `InlineCompletionItemProvider` 接口触发:

```
public async provideInlineCompletionItems(  
  document: vscode.TextDocument,  
  position: vscode.Position,  
  context: vscode.InlineCompletionContext,  
  token: vscode.CancellationToken,  
)
```

工具调用统计

工具	调用次数	调用原因
Gre p	4次	搜索 <code>autocomplete</code> 、 <code>trigger.*completion</code> 、 <code>debounce</code> 、 <code>DEFAULT_AUTOCOMPLETE_OPTS</code> 定位相关文件和配置
Glo b	1次	查找 <code>autocomplete</code> 目录下所有文件, 了解模块结构
Rea d	6次	读取核心文件: <code>completionProvider.ts</code> (VS Code 入口)、 <code>CompletionProvider.ts</code> (核心逻辑)、 <code>prefiltering/index.ts</code> (预过滤)、 <code>AutocompleteDebounce.ts</code> (防抖)、 <code>shouldCompleteMultiline.ts</code> (多行决策)、 <code>HelperVars.ts</code> (上下文构建)

总计: 11次工具调用, 全面分析了自动补全的触发机制、防抖策略、过滤条件和配置参数。

Claude Tokens	GPT-4o Tokens	Gemini Tokens	Characters
33,229	25,211 (-24.1%)	31,849 (-4.2%)	94,643

聊天 JOYCODE

查找"自动补全功能的触发逻辑" 完成任务的时候列出各个工具调用的次数, 以及每次调用的原因

编码

已深度思考 >

我来帮你查找"自动补全功能的触发逻辑"。让我使用搜索工具来定位相关代码。

CSR上下文引擎 autocomplete trigger logic

已找到20条结果

已深度思考 >

CSR上下文引擎 trigger(debounce|provideInlineCompletionItems

已找到20条结果

已深度思考 >

JoyCode 读取了多个文件:

工具调用统计

- `term_sparse_search`: 1次 - 初步搜索自动补全触发逻辑相关代码
- `grep_search`: 2次 - 精确搜索触发、防抖和预过滤相关代码
- `read_file`: 7次 - 读取核心文件详细内容

Claude Tokens	GPT-4o Tokens	Gemini Tokens	Characters
20,319	16,332 (-19.6%)	18,546 (-8.7%)	58,377

案例分析

项目二

<https://github.com/microsoft/vscode>

文件数: 4.2w+

代码行数: 1000w+

Prompt: 找出 VSCode 中所有参与"工作区信任验证"流程的组件, 包括触发点、验证逻辑、UI 提示



竞品对比

找出 VSCode 中所有参与"工作区信任验证"流程的组件，包括触发点、验证逻辑、UI 提示；完成任务的时候列出各个工具调用的次数，以及每次调用的原因

我来帮你找出 VSCode 中"工作区信任验证"流程的所有组件。让我开始搜索相关代码。

Searched for 11 patterns, read 12 files (ctrl+o to expand)

现在我已经收集了足够的信息。让我为你总结 VSCode 中工作区信任验证流程的所有组件。

VSCode 工作区信任验证流程组件总结

一、核心服务架构

1. 平台层接口 (`src/vs/platform/workspace/common/workspaceTrust.ts`)

```
graph TD
    A[calculateWorkspaceTrust() — 计算信任状态] --> B[_onDidChangeTrust.fire() — 触发信任变更事件]
    B --> C[WorkspaceTrustContextKeys — 更新上下文键]
    B --> D[WorkspaceTrustUXHandler — 更新 UI 指示器]
```

八、工具调用统计

工具: Grep
调用次数: 8 次
调用原因: 搜索
`workspaceTrust`、`IWorkspaceTrustRequestService`、`requestWorkspaceTrust`、`requestOpenFilesTrust`、`MANAGE_TRUST_COMMAND_ID`、`WorkspaceTrustContext`、`security.workspace.trust`
等关键词定位相关代码

工具: Read
调用次数: 10 次
调用原因: 读取核心文件: `workspaceTrust.ts` (平台层和工作台层)、`workspace.contribution.ts`、`extensionManifestPropertiesService.ts`、`terminalInstance.ts`、`editorService.ts`、`abstractTaskService.ts`、`debugService.ts`、`testServiceImpl.ts`、`notebookExecutionServiceImpl.ts`、`workspaceTrustEditor.ts`

工具: Glob
调用次数: 0 次
调用原因: 本次任务未使用 (主要用 Grep 搜索内容)

总计: 18 次工具调用 (8 次 Grep + 10 次 Read)

聊天 JOYCODE

找出 VSCode 中所有参与"工作区信任验证"流程的组件，包括触发点、验证逻辑、UI 提示；完成任务的时候列出各个工具调用的次数，以及每次调用的原因

编码

已深度思考 >

我会帮你找出 VSCode 中参与"工作区信任验证"流程的所有组件。让我先搜索相关代码。

Q CSR上下文引擎 workspace trust 已找到20条结果 ✓

已深度思考 >

```
graph LR
    A[_trustTransitionManager.participate() — 执行过渡参与者] --> B[_onDidChangeTrust.fire() — 触发信任变更事件]
```

五、工具调用统计

本次分析共进行了 15 次工具调用:

inverted_index_search (1次)

- 搜索 "workspace trust" - 定位工作区信任相关文件

grep_search (6次)

- 搜索 "WorkspaceTrustManagementService" - 查找信任管理服务使用点
- 搜索 "requestWorkspaceTrust" - 查找信任请求调用点
- 搜索 "WorkspaceTrustRequestService" - 查找信任请求服务引用
- 搜索 "workspaceTrustDialog" - 查找对话框相关代码
- 搜索 "onDidChangeTrust" - 查找信任状态变更监听器
- 搜索 "requestWorkspaceTrustOnStartup" - 查找启动时信任初始化

read_file (8次)

- 读取接口定义文件 - 了解服务接口契约 2-5. 读取信任服务实现 - 理解验证逻辑 6-8. 读取UI贡献点文件 - 了解对话框和提示处理
- 读取信任编辑器 - 了解管理界面
- 读取扩展启用过渡参与者 - 了解扩展信任联动

Claude Tokens

52,652

GPT-4o Tokens

35,236

(-33.1%)

Gemini Tokens

49,055

(-6.8%)

Characters

131,505

Claude Tokens

46,443

GPT-4o Tokens

34,793

(-25.1%)

Gemini Tokens

40,195

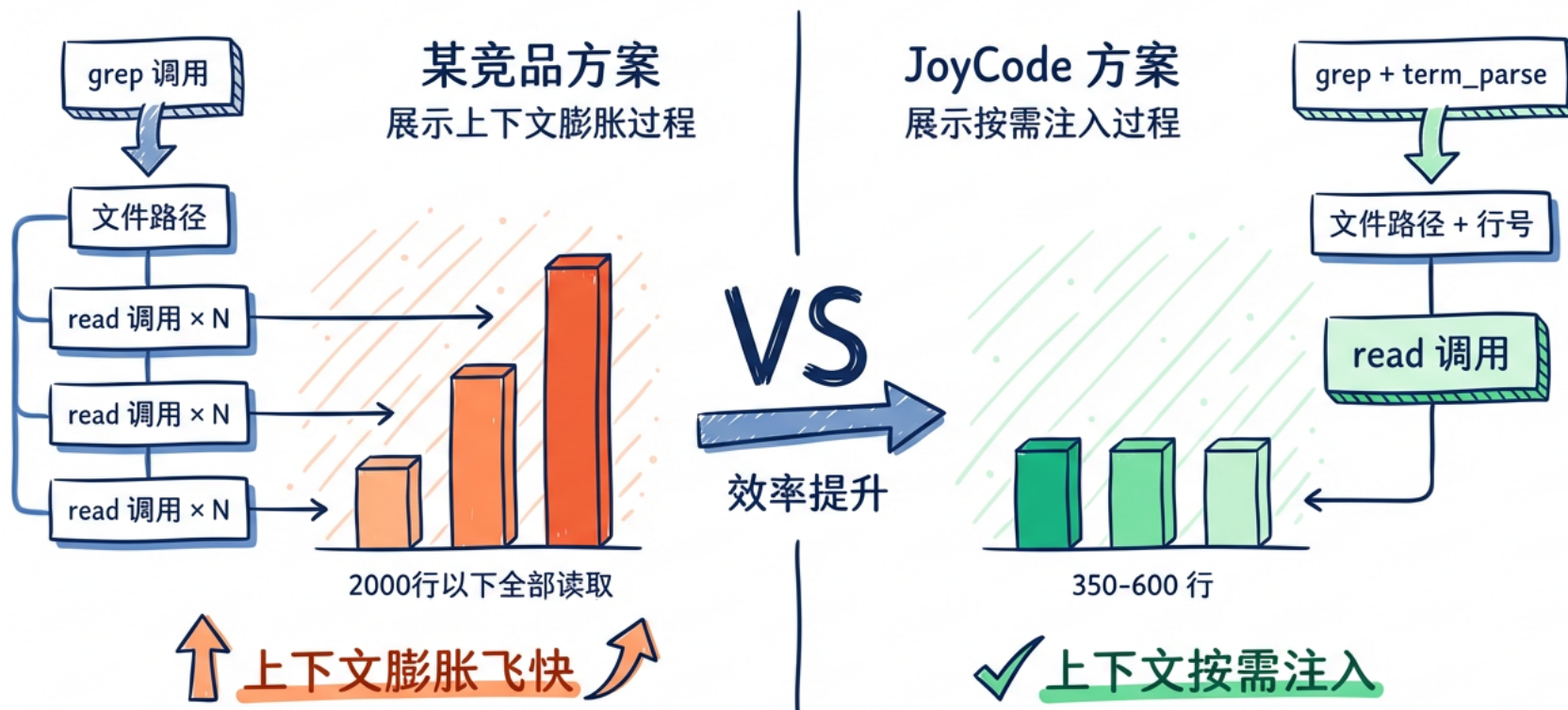
(-13.5%)

Characters

121,779

竞品对比-为什么 token 消耗差距会这么大

上下文管理对比 - Comparison



团队智能体

上下文管理

检索工具、短期记忆等是核心

内置智能体

实践经验沉淀的各类内置智能体，如规约编程、问题修复、前端页面设计等



丰富地扩展工具

MCP、Skills、BashTool

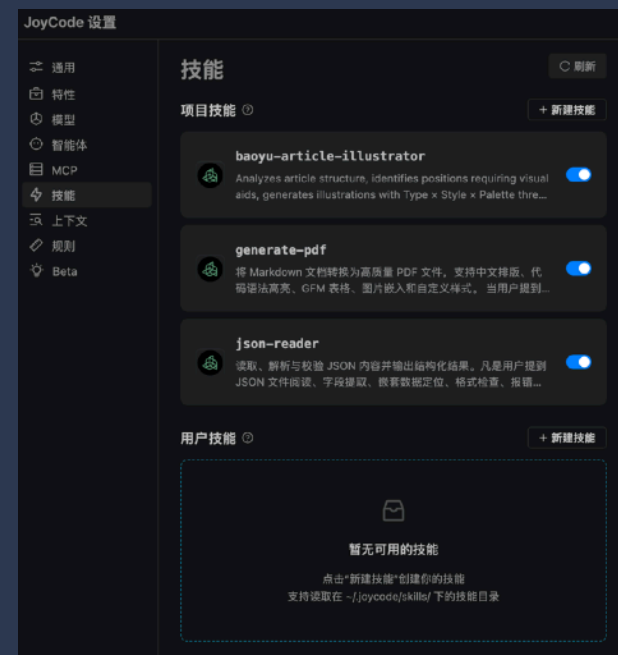
自定义智能体

用户自定义配置智能体，且支持相互调度

多层次智能体协作体系

智能体团队

丰富地扩展能力、内置智能体、自定义智能体、智能体自由调度切换



智能体团队 – 自主调度

基于智能体团队的 Harness Engineer 实践：协调智能体、计划智能体、编码智能体、评估智能体

协调智能体

作为中央协调器，自主驱动整个开发生命周期。核心职责包括：

- 无限循环调度Planner、Generator、Evaluator子智能体
- 管理Sprint状态机和状态转换
- 维护所有Sprint的执行进度
- 确保所有Sprint完成后才进行最终验收

Orchestrator

计划智能体

负责将用户简短的1-4句话需求扩展为完整的产品规格说明书和Sprint计划。核心职责包括：

- 需求分析与功能拆解
- 设计技术架构方案
- 制定最多5个Sprint的迭代计划
- 定义每个功能的验收标准
- 为Generator和Evaluator提供工作基准

Planner

生成智能体

负责根据产品规格和Sprint计划逐步实现功能代码。核心职责包括：

- 与Evaluator协商Sprint合同
- 按合同实现所有功能
- 进行自我评估和代码审查
- 确保每个Sprint产出可运行的增量功能
- 每次Sprint完成后交付给Evaluator进行QA验证

Generator

评估智能体

作为独立的质量守门人，负责审查和验证。核心职责包括：

- 审查Sprint合同的完整性和可测试性
- 使用Playwright进行实际浏览器测试验证
- 评估设计质量、原创性、功能完整性、工艺水平、代码质量等维度
- 决定PASS/FAIL或APPROVE/REJECT
- 拥有项目最终验收的独有权 (FINAL_APPROVED/FINAL_REJECTED)

Planner

智能体团队

自主运行 6.5 小时。

每个子代理拥有独立的上下文窗口，有效解决上下文窗口膨胀的问题。

The screenshot displays a development environment with a file explorer on the left, a central preview window, and a right-hand chat interface.

File Explorer (Left): Shows a project structure under 'PROJECT5' with subfolders 'joycode', 'joytasks', 'plans', and 'sprints'. The 'sprints' folder is expanded, showing files like 'sprint-1-contract-propos...', 'sprint-1-contract-review...', 'sprint-1-qa-report.md', 'sprint-1-self-review.md', 'sprint-2-contract-propos...', 'sprint-2-contract-review...', 'sprint-2-contract.md', 'sprint-2-qa-report.md', 'sprint-2-self-review.md', and 'orchestration-status.md'. Other folders include 'rules', 'retro-game-maker', 'dist', 'node_modules', 'public', 'src', '.gitignore', 'eslint.config.js', 'index.html', 'package-lock.json', 'package.json', 'README.md', 'tsconfig.app.json', 'tsconfig.json', 'tsconfig.node.json', and 'vite.config.ts'.

Preview Window (Center): Displays a document titled 'Preview sprint-1-contract-review.md'. The content is a 'Sprint 2 QA Report'.

Sprint 2 QA Report

Overview

- Sprint Number:** 2
- Theme:** Sprite Editor & Assets - 精灵编辑器与资产管理系统
- Test Date:** 2026-04-06
- Tester:** Evaluator Agent
- Test Method:** Playwright Browser Testing

Scores

Dimension	Score	Threshold	Status
Design Quality	7/10	6	PASS
Originality	6/10	5	PASS
Product Depth	6/10	6	PASS
Craftsmanship	6/10	5	PASS
Functionality	6/10	7	FAIL
Code Quality	7/10	5	PASS

Overall: FAIL - Functionality score below threshold

Contract Test Criteria Verification

Sprint 2 QA Report

- Overview
- Scores
- Contract Test Criteria Verification

Critical Issues Found

- Issue 1: 精灵保存功能不工作 (Critical - BLOCKER)
- Issue 2: 自定义调色板创建UI缺失 (Major)
- Issue 3: RGB/HEX颜色输入UI缺失 (Major)
- Issue 4: 调色板导入导出功能缺失 (Major)
- Issue 5: 标签功能缺失 (Minor)

Performance Test Results

- Features Verified Working
- Features Not Working / Missing
- Decision: FAIL

Chat Interface (Right): A chat window titled 'JOYCODE' with a sub-header '资产库功能缺失'. It contains a list of verified features and a list of issues that must be fixed to pass verification.

Verified Normal Function:

- ✓ 精灵编辑器基本绘图 (画笔、橡皮擦、填充)
- ✓ 工具快捷键 (B/E/G/I/M)
- ✓ 内置调色板切换 (NES、Game Boy、CGA、PICO-8)
- ✓ 撤销/重做 (按钮和Ctrl+Z/Ctrl+Y)
- ✓ 画布缩放
- ✓ 动画编辑器创建
- ✓ 资产库文件夹创建和搜索

必须修复才能通过验收

- 修复精灵保存到IndexedDB的功能
- 添加自定义调色板创建UI
- 添加RGB/HEX颜色输入
- 添加调色板导入/导出
- 完善选区操作功能

详细报告:

.joycode/joytasks/sprints/sprint-2-qa-report.md

Orchestrator

已成功调度 Evaluator 子代理进行 Sprint 2 的第二轮 QA 验收测试。Evaluator 将使用 Playwright 进行实际的浏览器测试，验证所有修复是否有效，并决定 Sprint 2 是否通过验收。

您可以随时取消并调整任务 取消任务

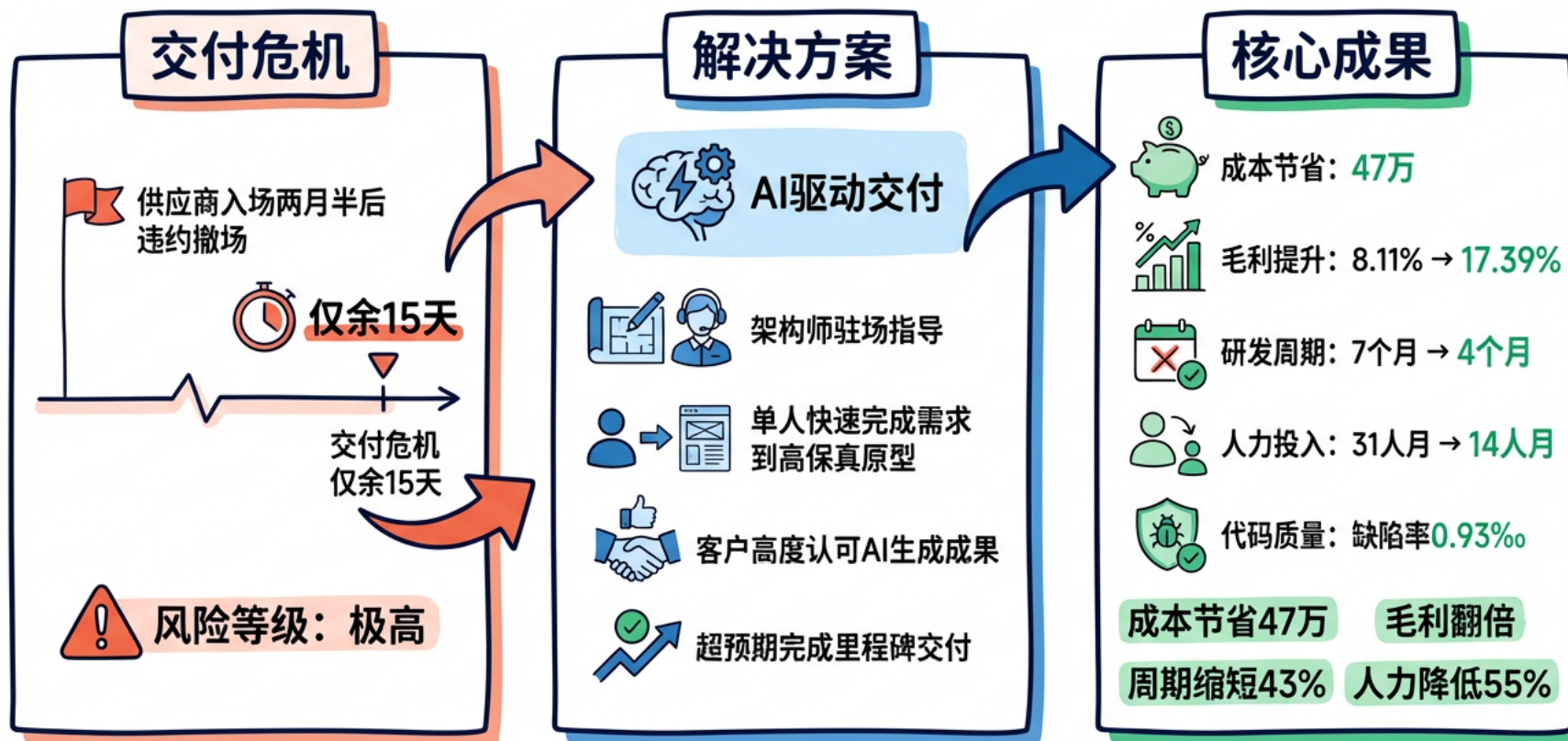
> 1 变更文件 拒绝 接受

通过 "@" 添加上下文，"/" 使用快捷指令，"↑↓" 切换历史输入，"⌘+↵" 换行

Footer: JoyCode, Prettier

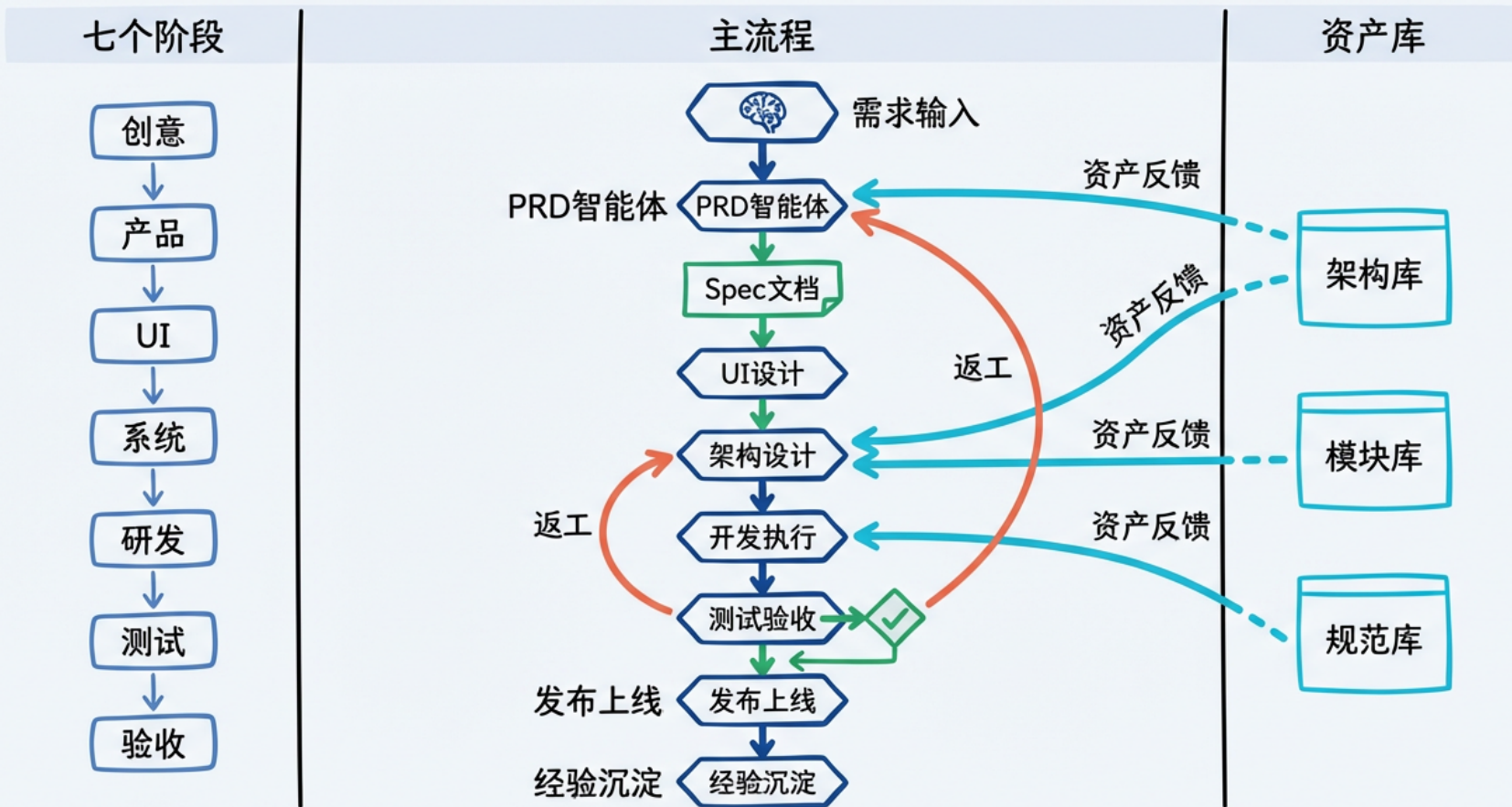
真实场景案例-某能源企业风险防控项目交付

AI交付提效项目背景 - Timeline Story



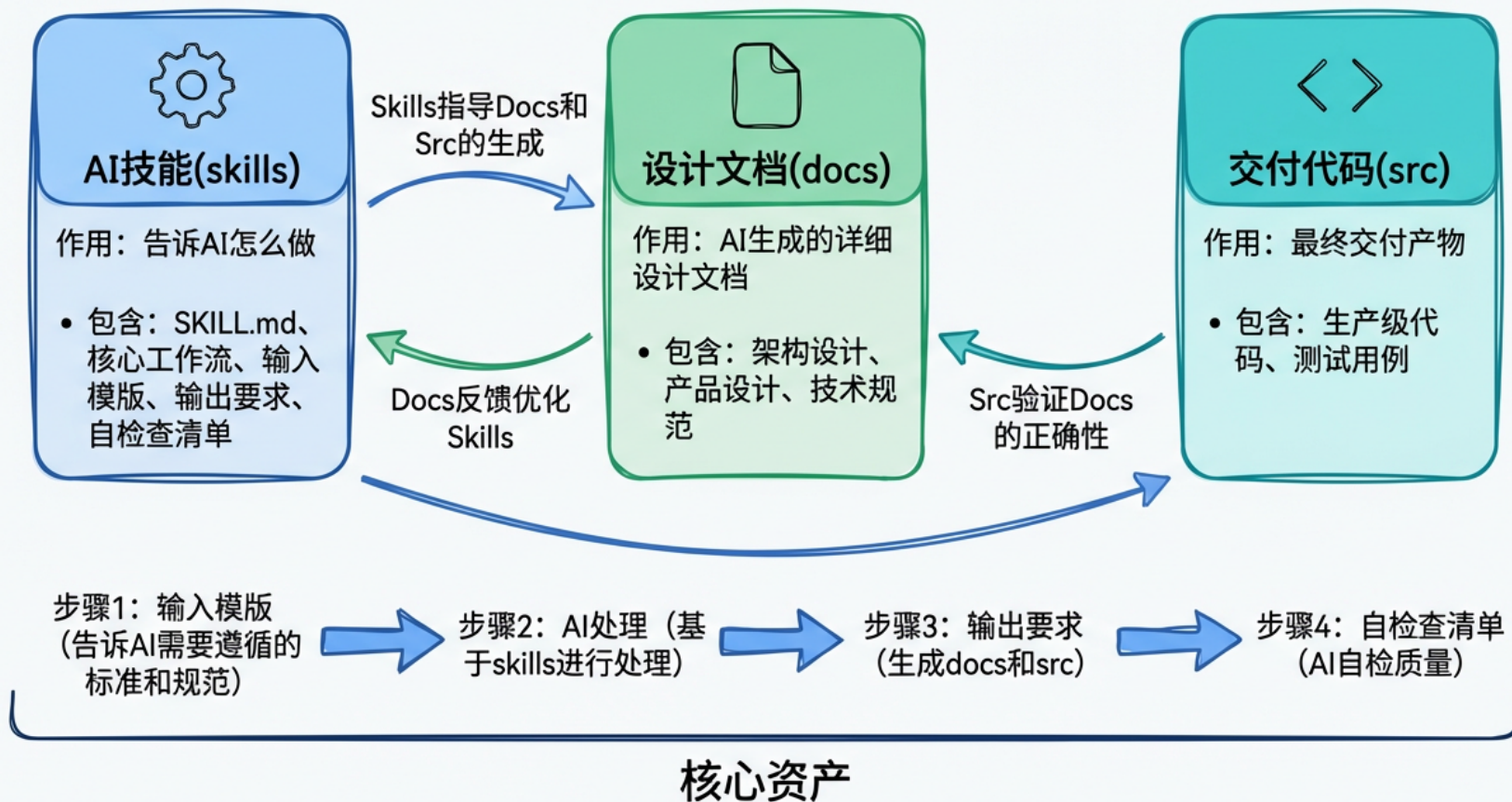
某能源企业供应商风险防控项目交付

AI驱动软件工程全生命周期交付流程

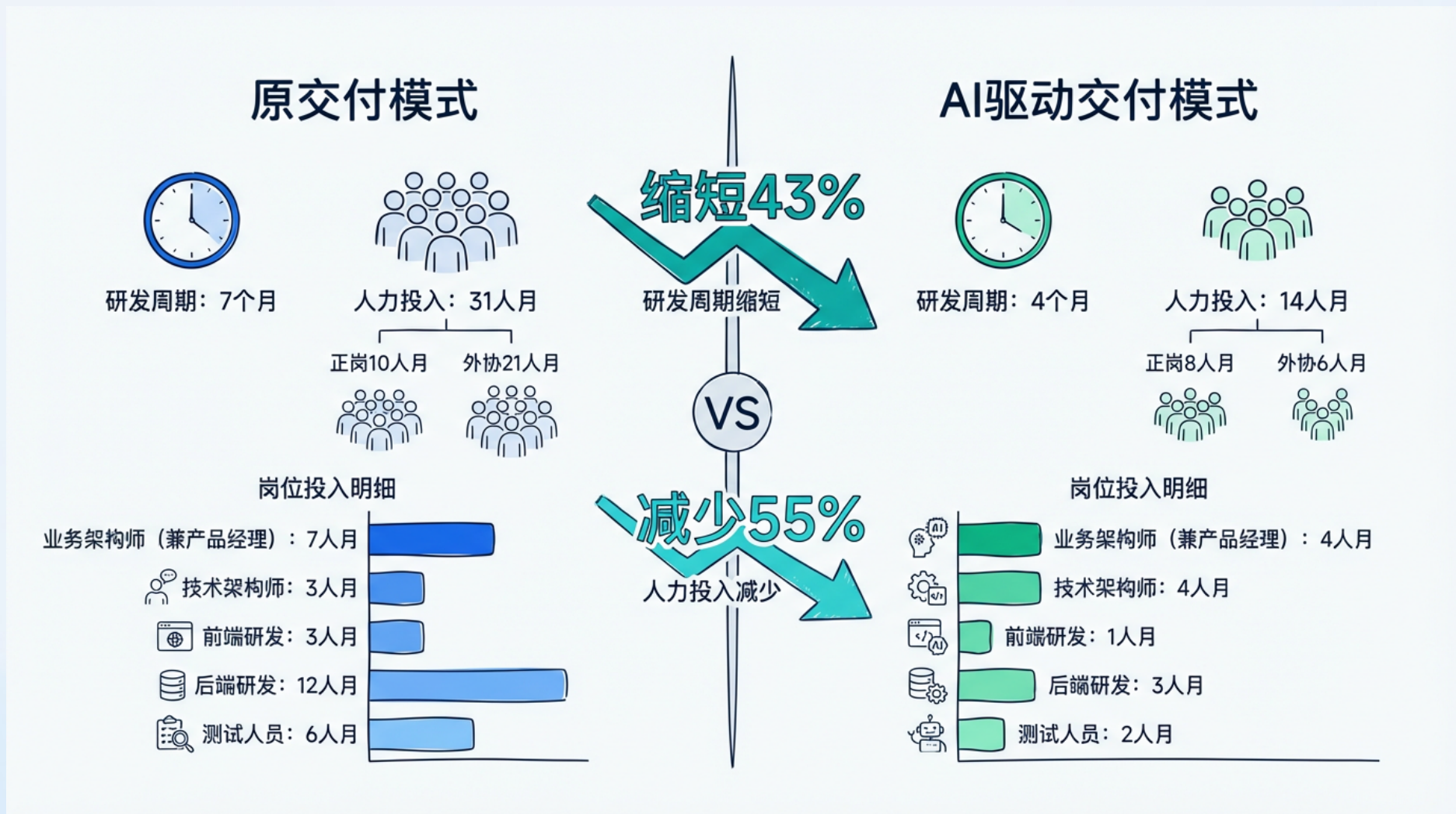


某能源企业供应商风险防控项目交付

AI交付流程框架 - Conceptual Framework



某能源企业供应商风险防控项目交付

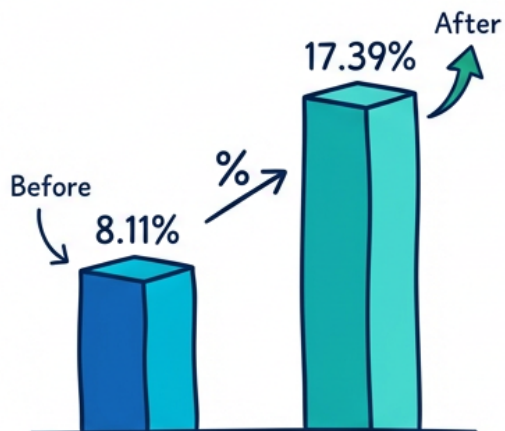


某能源企业供应商风险防控项目交付

项目预算总览



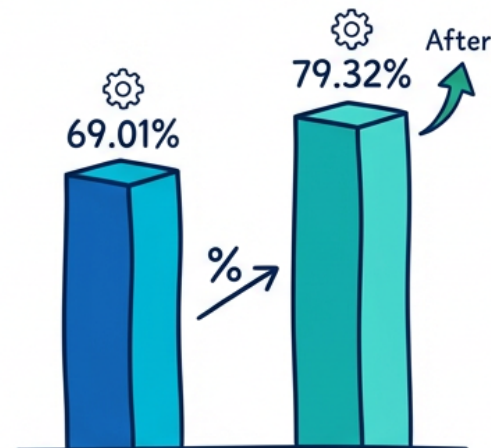
毛利提升: 8.11% → 17.39%



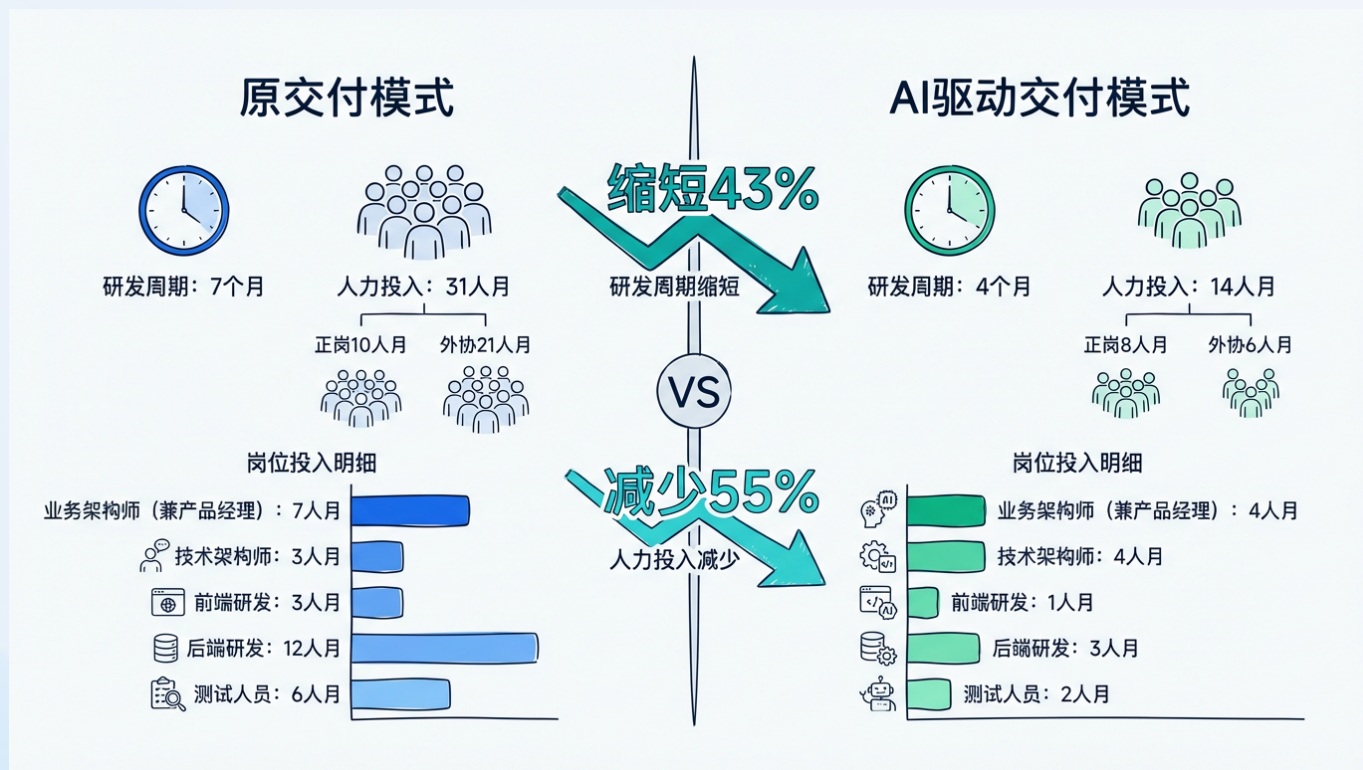
节省成本: 约50万



自研占比: 69.01% → 79.32%



未来目标



1 人月 =>
15 人天?
1人天?

总结回顾

JoyCode 是如何应对挑战的

业务场景复杂

知识来源多样化

在线文档、本地多模态、远程智能体编排、自研组件、UI 设计...

用户画像丰富

自定义智能体

系统提示词、指令、规则、工具、skills、mcp 自由组合，多 Agent 自由调度

复杂长任务

抑制上下文窗口膨胀

以检索为核心，按需加载；
子智能体在独立上下文窗口运行，仅将 Summary 回传；其他机制保障

超大代码仓

仓库级理解工具

CodeWiki 代码即知识的高阶知识；代码图谱 提供仓库级别的理解工具；多路索引保证性能和召回率

03 未来趋势



未来趋势

上下文工程仍是核心

在有限的上下文窗口中注入最准确的信息，仍是我们的目标。

在数百上千次工具调用之后，保证上下文仍然可以保持稳定且高效地运行。

Harness Engineer 不会彻底消失

模型能力不发生跃迁，
Harness 不会消失。

提供尽可能全面且高效的工具、Agent 驱动机制，让模型可以灵活自主地进行选择、组装调用，更好地完成任务。

大展宏“图”

从 context7、git nexus、
graphti 再到 supermemory

图技术在 AI Coding 领域的应用会越来越广泛，甚至可能会成为不同产品之间的核心竞争力。

JoyCode 算法团队热情招募-天才计划

JoyCode 算法团队专注大语言模型在软件开发全生命周期的深度应用，支持代码补全、智能编辑等核心AI功能优化。

课题聚焦代码智能化：基于Code LLM构建代码补全、生成、优化等核心能力，打造世界一流产品。

诚邀对 AI Coding 充满热情的你，共同定义下一代智能化开发范式。

欢迎投递：xuxiang.118@jd.com

THANKS

大模型正在重新定义软件

Large Language Model Is Redefining The Software



AiCon

全球人工智能开发与应用大会

上海站

探索 AI 应用边界

Explore the limits of AI applications

2026 年 6 月 26-27 日 / 上海张江科学会堂



参会咨询

专题：世界模型与多模态智能突破

专题出品人：朱政

极佳视界 / 联合创始人 & 首席科学家



专题：Agent 系统架构与工程化实践

专题出品人：鲁浩楠

OPPO / 大模型算法负责人



专题：AI 原生数据工程

专题出品人：王彦辉

火山引擎 / 数智平台产品总监



专题：Agent 安全、评测与可信治理

专题出品人：马传雷（岳立）

支付宝 / 业务风险技术部负责人



专题：Agent 数据、记忆与运行时基础设施

专题出品人：邓亚峰

EverMind / CEO



专题：企业级研发体系的重构

专题出品人：沈浪

快手 / 研发效能负责人

